

Lista de exercícios da Sprint 1

Trilha IA & Dados - Portal do Patrocinador

Planeta Atlântida - Desafio GC 2026

SQL e modelagem relacional | Python e pandas | PostgreSQL e RLS | FastAPI e OpenAPI | Git e Docker Compose

O que tem aqui

124 exercícios em 5 partes, mais um mini-projeto integrador e uma autoavaliação conceitual.

Cada parte começa com os **conceitos** (para entender antes de fazer) e os exercícios sobem de dificuldade, do aquecimento ao desafio.

Os exercícios pedem que **você escreva o código**. Não há soluções prontas; há dicas em forma de pergunta e formas de conferir se acertou.

Arquivo de apoio: [midia_ficticia.xlsx](#) (dados fictícios e sujos de propósito, usado na Parte 2).

Como usar esta lista

- 1. Conceito primeiro, depois a mão na massa.** Leia a caixa de conceitos de cada parte, tente explicar em voz alta com suas palavras e só então comece os exercícios.
- 2. Tente sozinho antes de olhar qualquer coisa.** As "Dicas" são perguntas para te destravar, não respostas. O guia técnico da equipe tem exemplos de código; use-os para **conferir** depois de tentar, não para copiar.
- 3. Largura primeiro, profundidade depois.** Faça os níveis N1 a N3 de todas as partes (isso cobre a Semana 1 do seu plano), e só então volte para os N4 e N5. Assim você descobre cedo onde estão as lacunas, que é a ideia do mini-projeto.
- 4. Mantenha um diário de erros.** Todo erro que você provocar de propósito ou sem querer vale ouro. Anote com a estrutura da última página.
- 5. Quando travar.** Se passar de uns 20 minutos no mesmo ponto: releia a documentação indicada, reduza o problema ao menor exemplo possível e, se continuar travado, mande para mim o que tentou e a mensagem de erro completa. Posso explicar o conceito e revisar seu código sem entregar a solução de bandeja.
- 6. Conferência.** Resultados numéricos esperados ficam nos Apêndices B e C, para você não ser influenciado antes de tentar. Os itens são citados em "Como conferir".

Nível	Significado	Quantidade
N1 - Aquecimento	Reproduzir algo simples e ver funcionar	17
N2 - Básico	Aplicar um conceito isolado	26
N3 - Intermediário	Combinar conceitos e tratar casos de borda	37
N4 - Avançado	Resolver problemas parecidos com os do projeto	30
N5 - Desafio	Integrar várias peças, com teste ou medição	14

Sugestão de ordem	O que fazer
Dias 1-2	Parte 1, blocos A e B (modelagem e DDL). Em paralelo: Git GD-01 a GD-04 e Docker GD-11 e GD-12.
Dias 3-4	Parte 1, blocos C e D (consultas e índices), até o nível N3.
Dias 5-6	Parte 2 (pandas), N1 a N3.
Dia 7	Parte 3, blocos A a C até N3 (schemas, roles, primeiro contato com RLS).
Semana 2	Parte 4 (FastAPI/OpenAPI) até N3, Parte 5 completa até N3, depois voltar para os N4 e N5 de todas as partes.
Ao longo de tudo	Mini-projeto (Parte 6) como fio condutor e autoavaliação (Parte 7) no fim de cada semana.

Isto é só uma sugestão de ritmo; ajuste ao seu tempo. O que importa é a ordem de dificuldade.

Sumário

Como usar esta lista	2
Sumário	3
Parte 1 - SQL e modelagem relacional	5
Bloco A - Modelagem no papel (antes de qualquer SQL)	5
Bloco B - DDL no PostgreSQL	6
Bloco C - Consultas (use o Dataset A do Apêndice A)	6
Bloco D - Índices, plano de execução e transações	8
Parte 2 - Python para dados (pandas)	9
Preparação: o arquivo midia_ficticia.xlsx	9
Bloco A - Ler e explorar	9
Bloco B - Limpeza	9
Bloco C - Agregações e KPIs	10
Bloco D - Organização, testes e exportação	10
Bloco E - Desafios integradores	11
Parte 3 - PostgreSQL na prática	12
Bloco A - Schemas e roles	12
Bloco B - Views, views materializadas e JSONB	12
Bloco C - Row-Level Security	13
Parte 4 - APIs REST, FastAPI, Pydantic e OpenAPI	15
Bloco A - HTTP e primeiros endpoints	15
Bloco B - Validação e estrutura	15
Bloco C - Contract-first, jobs e testes	16
Bloco D - Integração com banco e contexto de marca	16
Parte 5 - Git e Docker Compose	18
Bloco A - Git	18
Bloco B - Docker e Docker Compose	19
Parte 6 - Mini-projeto integrador (Semana 1)	20
Parte 7 - Autoavaliação conceitual	20
Modelo de diário de erros	21
Apêndice A - Dataset A (script de carga para os exercícios SQL)	22

Apêndice B - Resultados esperados (SQL)	24
Apêndice C - Resultados esperados (pandas, midia_ficticia.xlsx)	26

Parte 1 - SQL e modelagem relacional

Por que importa: É a base de tudo no projeto: o modelo de dados que a equipe Java e a de Dados vão construir juntos na Sprint 1 sai daqui, e cada KPI do relatório é uma consulta SQL.

Conceitos antes de começar

Modelo relacional. Dados vivem em tabelas (linhas = registros, colunas = atributos). Uma **chave primária (PK)** identifica cada linha de forma única; uma **chave estrangeira (FK)** aponta para a PK de outra tabela e impede referências a coisas que não existem (integridade referencial).

Cardinalidade. 1:N (uma cota tem várias marcas) vira uma FK no lado N. N:N (usuários x marcas) não cabe em uma coluna só: vira uma **tabela associativa** com duas FKs, e atributos do relacionamento (papel, validade) moram nela.

Normalização. Objetivo: cada fato é guardado em um único lugar, evitando anomalias de atualização (mudar o nome da cota em 20 linhas), de inserção e de exclusão. **1FN:** valores atômicos, sem listas dentro de células. **2FN:** nenhum atributo depende de só parte da chave composta. **3FN:** nenhum atributo não-chave depende de outro atributo não-chave (dependência transitiva).

Consultas. **WHERE** filtra linhas antes de agrupar; **GROUP BY** colapsa linhas em grupos; **HAVING** filtra grupos depois da agregação. **INNER JOIN** mantém só pares que casam; **LEFT JOIN** mantém todas as linhas da esquerda, preenchendo com **NULL** quando não há par. **NULL** significa 'desconhecido': **NULL = NULL** não é verdadeiro, use **IS NULL**.

Índices. Estrutura auxiliar (geralmente B-tree) que evita varrer a tabela inteira para achar linhas. Custo: ocupa espaço e deixa INSERT/UPDATE um pouco mais lentos. Nem todo filtro precisa de índice; quem decide é o planejador, e você confere com **EXPLAIN ANALYZE**.

Leituras: Documentação oficial: [postgresql.org/docs/16](https://www.postgresql.org/docs/16) (capítulos 'The SQL Language' e 'Data Definition'). Visualizador de MER: dbdiagram.io ou Mermaid.

Bloco A - Modelagem no papel (antes de qualquer SQL)

SQL-01

N1 -
Aquecimento

A partir do texto 'uma marca pertence a uma cota; usuários acessam marcas; marcas enviam arquivos e têm itens de mídia', liste as **entidades** e escreva 3 a 5 atributos para cada uma. Marque quais atributos poderiam ser PK.

Como conferir: Você deve chegar a pelo menos: marca, cota, usuário, arquivo, item de mídia.

SQL-02

N1 -
Aquecimento

PK natural x PK substituta: para **marca**, compare usar o CNPJ como PK com usar um **uuid** gerado. Escreva 2 vantagens e 2 desvantagens de cada em poucas linhas.

Dica: Pense: o CNPJ pode mudar de formato? Pode ser nulo em algum caso? E em uma URL, qual expõe mais informação?

SQL-03

N2 - Básico

Desenhe o MER de **brand**, **tier**, **user**, **file** e **media_item** (dbdiagram.io ou Mermaid), indicando PKs, FKs e a cardinalidade de cada relação.

Como conferir: Toda relação 1:N tem a FK do lado N. Nenhuma tabela deve ter coluna que guarde lista de ids.

SQL-04

N2 - Básico

Usuário e marca são N:N (uma agência atende várias marcas, uma marca tem vários usuários). Desenhe a tabela associativa. Quais atributos pertencem a ela e não a **user** nem a **brand**? Qual deveria ser a PK dela?

Como conferir: Pelo menos: papel (role) e período de validade. Duas opções de PK: composta (**user_id**, **brand_id**) ou substituta + **UNIQUE**.

SQL-05 N3 - Intermediário	Normalização guiada. Dada a tabela plana abaixo, (a) liste as dependências funcionais, (b) aponte as anomalias de atualização, inserção e exclusão e (c) decomponha até a 3FN. <code>acesso(marca, nome_cota, detalhe_cota, email_usuario, nome_usuario, papel, arquivo, tipo_arquivo)</code> Dica: Pergunte a cada coluna: 'quem determina você?'. Ex.: marca determina nome_cota? E nome_cota determina detalhe_cota? Como conferir: Você deve terminar com pelo menos 5 tabelas, sem coluna repetida entre elas além das FKs.
SQL-06 N3 - Intermediário	Na tabela <code>brand(id, name, tier_code, tier_name, detail_level)</code> aponte a violação da 3FN, explique qual anomalia ela causa em um exemplo concreto e corrija.
SQL-07 N3 - Intermediário	Conceitual: o guia usa 3FN no schema <code>core</code> e um star schema (que repete dados de propósito) no <code>dw</code>. Escreva em 4 a 6 linhas por que desnormalizar é aceitável no segundo caso, mas não no primeiro. Dica: Pense em quem escreve, quem lê e com que frequência.

Bloco B - DDL no PostgreSQL

SQL-08 N1 - Aquecimento	Suba um PostgreSQL 16 em container (<code>docker run</code> com <code>postgres:16</code>, senha por variável de ambiente, porta 5432). Conecte pelo <code>psql</code> e por um cliente gráfico (DBeaver ou pgAdmin). Execute <code>SELECT version();</code>. Dica: Vale como aquecimento para a parte de Docker. Como conferir: A saída mostra PostgreSQL 16.x.
SQL-09 N1 - Aquecimento	Crie um banco <code>portal_dev</code> e, dentro dele, um schema <code>core</code>. Liste os schemas pelo <code>psql</code> (comando de barra) e por uma consulta ao catálogo <code>information_schema</code>.
SQL-10 N2 - Básico	Escreva o DDL de <code>ref.tier</code> (smallint PK, <code>code</code> único, <code>name</code>, <code>detail_level</code>) e <code>core.brand</code> (PK uuid com <code>gen_random_uuid()</code>, <code>name</code> NOT NULL, FK para tier). Insira 3 cotas e 5 marcas. Como conferir: Inserir uma marca com <code>tier_id</code> inexistente deve falhar com erro de chave estrangeira.
SQL-11 N2 - Básico	Constraints. Crie <code>core.courtesy_allocation</code> com <code>qty_contracted</code>, <code>qty_used</code> e um <code>CHECK (qty_used <= qty_contracted)</code>. Provoque de propósito 4 violações (NOT NULL, UNIQUE, CHECK, FK) e copie a mensagem de erro de cada uma no seu diário de erros. Como conferir: Quatro erros diferentes, cada um nomeando a constraint violada.
SQL-12 N3 - Intermediário	Crie <code>core.user</code> e <code>core.user_brand_access</code> (N:N). Teste o comportamento de <code>ON DELETE RESTRICT</code> e <code>ON DELETE CASCADE</code> apagando uma marca com vínculos. Em que situação do portal cada um faz sentido? Dica: Lembre que o portal faz exclusão lógica (<code>deleted_at</code>) em vez de apagar.
SQL-13 N3 - Intermediário	Crie <code>core.media_item</code> com: tipo de mídia, métricas <code>bigint</code> anuláveis, <code>created_at</code> com default <code>now()</code> e <code>deleted_at</code>. Faça duas versões do tipo de mídia: com <code>CREATE TYPE ... AS ENUM</code> e com tabela de referência + FK. Compare em 3 linhas qual é mais fácil de evoluir. Dica: O que acontece quando surge um novo tipo de mídia daqui a seis meses?
SQL-14 N4 - Avançado	Reúna o DDL de todo o MER do SQL-03 em um único arquivo <code>V001_criar_core.sql</code>. Rode em um banco vazio sem erros. Depois destrua tudo (<code>DROP SCHEMA ... CASCADE</code>) e rode de novo. O arquivo funciona nas duas vezes? Dica: Ordem importa: tabelas referenciadas primeiro. Como conferir: Você consegue recriar o banco do zero com um único comando.
SQL-15 N4 - Avançado	Unicidade natural: crie uma constraint <code>UNIQUE (brand_id, aired_date, vehicle, title)</code> em <code>media_item</code>. Tente inserir a mesma linha duas vezes. Depois use <code>INSERT ... ON CONFLICT DO NOTHING</code> e <code>ON CONFLICT DO UPDATE</code> e explique a diferença de resultado. Isso é o que torna a carga idempotente.

Bloco C - Consultas (use o Dataset A do Apêndice A)

SQL-16 N1 - Aquecimento	Liste todas as marcas; depois só nome e setor; depois ordenadas por nome; depois as 3 primeiras.
SQL-17 N1 - Aquecimento	Em <code>media_item</code>: (a) só as de tipo DIGITAL; (b) veiculadas entre 10 e 17 de janeiro de 2026; (c) com <code>clicks</code> nulos; (d) com <code>clicks</code> não nulos e maiores que 5000. Dica: Tente escrever <code>clicks = NULL</code> e observe o resultado. Por que não funciona? Como conferir: Resultados em Apêndice B, item B1.
SQL-18 N2 - Básico	Liste cada marca com o nome da sua cota (<code>JOIN</code> entre <code>brand</code> e <code>tier</code>). Como conferir: 10 linhas.
SQL-19 N2 - Básico	Quantas marcas há por cota? Qual o total de impressões por tipo de mídia, do maior para o menor? Como conferir: Apêndice B, item B2.
SQL-20 N2 - Básico	Em <code>media_item</code>, compare <code>COUNT(*)</code>, <code>COUNT(clicks)</code>, <code>COUNT(impressions)</code> e <code>COUNT(DISTINCT brand_id)</code>. Explique em uma frase por que os quatro números diferem. Como conferir: Apêndice B, item B3.
SQL-21 N3 - Intermediário	Para todas as marcas, mostre o total de impressões, com 0 para quem não tem mídia (<code>LEFT JOIN + COALESCE</code>). Dica: Se virem menos de 10 linhas, você usou o tipo de join errado. Como conferir: 10 linhas; duas marcas com 0 (Apêndice B, item B4).
SQL-22 N3 - Intermediário	Liste as marcas sem nenhuma veiculação de duas formas: <code>LEFT JOIN ... IS NULL</code> e <code>NOT EXISTS</code>. Compare os planos com <code>EXPLAIN</code>. E com <code>NOT IN</code>: quando ele se comporta mal? Dica: Pense no que <code>NOT IN</code> faz se a subconsulta devolver um NULL. Como conferir: Apêndice B, item B4.
SQL-23 N3 - Intermediário	Com <code>HAVING</code>: marcas com 2 ou mais veiculações. Depois, só as que também somam mais de 1 milhão de impressões. Explique por que essa segunda condição não pode ir no <code>WHERE</code>.
SQL-24 N3 - Intermediário	CTR (cliques / impressões, em %, 2 casas) por marca, só para DIGITAL e SOCIAL. Trate divisão por zero com <code>NULLIF</code>. Reflita: por que calcular CTR sobre TV distorceria o resultado? Dica: Cuidado com inteiros: divisão de inteiros trunca. Force numeric. Como conferir: Apêndice B, item B5.
SQL-25 N3 - Intermediário	Percentual de cortesias usadas por marca (soma usada / soma contratada) e, separadamente, as combinações marca x área com 90% ou mais de uso. Como conferir: Apêndice B, item B6.
SQL-26 N4 - Avançado	Com CTE (<code>WITH</code>): marcas cujo total de impressões está acima da média dos totais por marca (entre as que têm mídia). Dica: Primeiro CTE: total por marca. Segundo: média. Depois compare. Como conferir: Apêndice B, item B7.
SQL-27 N4 - Avançado	Janela: ranking de marcas por impressões dentro de cada cota usando <code>RANK()</code> e <code>PARTITION BY</code>. Troque por <code>ROW_NUMBER()</code> e <code>DENSE_RANK()</code> e explique a diferença com um empate criado por você. Como conferir: Apêndice B, item B8.
SQL-28 N4 - Avançado	Impressões por semana (<code>date_trunc('week', ...)</code>) e impressões acumuladas (<code>SUM(...) OVER (ORDER BY ...)</code>). Como conferir: Apêndice B, item B9.
SQL-29 N4 - Avançado	Duplicatas: ache linhas repetidas pela chave natural (<code>GROUP BY ... HAVING COUNT(*) > 1</code>). Depois remova as repetidas mantendo uma de cada, usando <code>ROW_NUMBER()</code> em uma CTE. Faça dentro de uma transação e confira antes do <code>COMMIT</code>. Como conferir: Dois grupos duplicados; ao final 24 linhas (Apêndice B, item B10).

SQL-30 N5 - Desafio	Desafio: uma única consulta que devolva, por marca, o número de veiculações, impressões totais, CTR (só digital/social), % de cortesias usadas e a posição no ranking da cota. É o embrião do relatório parcial. Dica: Calcule cada bloco em uma CTE separada e junte por brand_id. Cuidado: juntar média com cortesias direto multiplica linhas e infla as somas. Por quê? Como conferir: Compare os números com os itens B5, B6 e B8.
SQL-31 N5 - Desafio	Pivot condicional: uma linha por marca e uma coluna para cada tipo (TV, RADIO, DIGITAL, SOCIAL, IMPRESSO, OOH) com a soma de impressões. Faça com CASE e com SUM(...) FILTER (WHERE ...). Como conferir: Para a marca 2, os valores devem bater com o item B11.

Bloco D - Índices, plano de execução e transações

SQL-32 N2 - Básico	Gere 1 milhão de linhas fictícias em uma cópia de media_item usando generate_series e random() (marcas de 1 a 40, datas em 2026, impressões aleatórias). Dica: Faça INSERT INTO ... SELECT ... FROM generate_series(1, 1000000). Como conferir: SELECT count(*) devolve 1.000.000.
SQL-33 N3 - Intermediário	Rode EXPLAIN (ANALYZE, BUFFERS) em WHERE brand_id = 7 antes e depois de criar um índice em brand_id. Anote o tipo de varredura (Seq Scan x Index Scan) e o tempo de cada um. Como conferir: O tempo cai e o plano muda. Se não mudar, rode ANALYZE na tabela.
SQL-34 N4 - Avançado	Índice composto (brand_id, aired_date): teste consultas filtrando só por brand_id, só por aired_date e pelos dois. Em qual delas o índice não ajuda? Explique a regra do prefixo mais à esquerda.
SQL-35 N4 - Avançado	Transações: abra duas sessões psql. Na primeira, BEGIN; UPDATE ...; sem COMMIT. Na segunda, leia a linha. O que você vê? Faça ROLLBACK e repita com COMMIT. Depois provoque um bloqueio (as duas atualizando a mesma linha) e observe quem espera. Dica: Isolamento padrão: READ COMMITTED.
SQL-36 N5 - Desafio	Desafio de desempenho: escreva a consulta do relatório (impressões por marca e semana) sobre 1 milhão de linhas, meça o tempo, proponha um índice, meça de novo e registre no diário a hipótese e o resultado. Tente também uma versão com view materializada (adianta a Parte 3). Como conferir: Registre os tempos: sem índice, com índice, materializada.

Parte 2 - Python para dados (pandas)

Por que importa: É o motor do serviço de dados: ler Excel, limpar, validar e calcular KPIs. O exercício do mini-projeto da Semana 1 depende de você ter essa parte afiada.

Conceitos antes de começar

DataFrame é uma tabela em memória; cada coluna é uma **Series** com um **dtype**. Operações **vetorizadas** (na coluna toda) são muito mais rápidas e legíveis do que laços linha a linha.

Valores ausentes. **NaN** (e **NaT** para datas, **pd.NA** nos tipos anuláveis como **Int64**) propagam em contas. **sum()** ignora ausentes por padrão, mas **a + b** com um ausente resulta em ausente.

groupby segue o padrão dividir-aplicar-combinar: divide em grupos, aplica uma agregação, junta o resultado. **merge** é o JOIN do SQL; **pivot_table** é o pivot.

Funções puras. Mesma entrada, mesma saída, sem alterar a entrada nem depender de estado externo. É o que torna o ETL testável: **limpar(df)** devolve um DataFrame novo e pode ser testada com 3 linhas de exemplo.

Arquivos reais são sujos. Cabeçalho fora da primeira linha, texto onde deveria ser número, datas em formatos misturados, caixa e espaços inconsistentes, duplicatas. A habilidade principal aqui é *descobrir* o problema antes de corrigir.

Leituras: pandas.pydata.org/docs, seções 'Getting started' e 'User Guide' (Working with missing data, Group by, Merge). Material de apoio: arquivo **midia_ficticia.xlsx**, entregue junto com este PDF.

Preparação: o arquivo **midia_ficticia.xlsx**

PY-00

N1 -
Aquecimento

Crie uma pasta de trabalho com ambiente virtual (**python -m venv .venv**), instale **pandas**, **openpyxl** e **pytest**, e coloque **midia_ficticia.xlsx** nela. O arquivo tem duas abas: **Midia** (dados sujos de propósito) e **Marcas** (cadastro correto de marcas e cotas). Os dados são fictícios. **Como conferir:** **python -c "import pandas; print(pandas.__version__)"** funciona.

Bloco A - Ler e explorar

PY-01

N1 -
Aquecimento

Leia a aba Midia com **pd.read_excel**. A primeira tentativa vai vir estranha. Descubra por quê e corrija com o parâmetro adequado.

Dica: Olhe as primeiras linhas do arquivo no próprio Excel.

Como conferir: Os nomes das colunas aparecem corretamente: Marca, Tipo de Mídia, Veículo...

PY-02

N1 -
Aquecimento

Explore: **shape**, **head**, **tail**, **info**, **dtypes**, **describe** e **isna().sum()**. Escreva 5 observações sobre o que parece errado.

Como conferir: Compare com Apêndice C, item C1 (número de linhas).

PY-03

N2 - Básico

Inventário de problemas. Sem corrigir nada ainda, liste todos os tipos de problema que encontrar (nomes, tipos, datas, números, nulos, duplicatas, valores impossíveis). Use **value_counts()**, **unique()**, **duplicated()** e filtros.

Dica: Meta: pelo menos 8 categorias. Use **df['Tipo de Mídia'].value_counts()** e olhe as colunas de data e impressões como texto.

Como conferir: Compare sua lista com o Apêndice C, item C2, só depois de tentar.

PY-04

N2 - Básico

Renomeie as colunas para **snake_case** (**marca**, **tipo_midia**, **veiculo**, **titulo**, **data_veiculacao**, **impressoes**, **cliques**, **alcance**) usando um dicionário de mapeamento.

Dica: Guarde o dicionário em uma constante, como o **MAP_COLUNAS** do guia.

PY-05

N2 - Básico

Seleção e filtro: (a) só **marca** e **impressoes**; (b) linhas com impressões acima de 1.000.000; (c) as 5 maiores por alcance. Note o que acontece com a coluna de impressões ainda como texto.

Bloco B - Limpeza

PY-06 N2 - Básico	Normalize tipo_midia até restarem só os 6 valores canônicos (TV, RADIO, DIGITAL, SOCIAL, IMPRESSO, OOH). Trate espaços, caixa e acentos (Rádio x RADIO). Dica: <code>str.strip</code>, <code>str.upper</code>, e <code>unicodedata</code> para remover acento. Como conferir: <code>df['tipo_midia'].unique()</code> devolve exatamente 6 valores (Apêndice C, item C3).
PY-07 N2 - Básico	Normalize marca e confira contra a aba Marcas: depois da limpeza o número de marcas distintas deve bater com o cadastro. Como conferir: Apêndice C, item C3.
PY-08 N3 - Intermediário	Converta data_veiculacao para datetime. A coluna mistura objetos de data, dd/mm/aaaa e aaaa-mm-dd. Use <code>pd.to_datetime</code> com <code>errors='coerce'</code> e liste as linhas que viraram NaT. Dica: Atenção ao <code>dayfirst</code>: 05/01 é 5 de janeiro ou 1 de maio? Como conferir: Número de datas inválidas em Apêndice C, item C4.
PY-09 N3 - Intermediário	Converta impressoes e cliques para inteiros. Há texto com ponto de milhar (1.250.000) e nulos. Use o tipo anulável Int64. Explique a diferença entre <code>int64</code> e Int64. Como conferir: Nenhuma exceção ao converter; nulos permanecem como <NA>.
PY-10 N3 - Intermediário	Valores impossíveis: transforme impressões negativas em nulo e crie uma coluna booleana cliques_invalidos para linhas com cliques maiores que impressões. Conte quantas linhas caem em cada regra. Como conferir: Apêndice C, item C4.
PY-11 N3 - Intermediário	Duplicatas pela chave natural (marca, data, veículo, título). Importante: faça depois de normalizar a marca. Quantas aparecem antes e depois da normalização? Por que a ordem importa? Como conferir: Apêndice C, item C5.
PY-12 N3 - Intermediário	Junte com a aba Marcas (<code>merge, how='left', indicator=True</code>) e verifique se alguma marca da mídia não existe no cadastro. Como conferir: Nenhuma linha com <code>_merge == 'left_only'</code> se a normalização estiver correta.

Bloco C - Agregações e KPIs

PY-13 N3 - Intermediário	groupby: impressões e cliques totais por marca e por tipo de mídia, ordenados do maior para o menor. Liste o top 3 de marcas. Como conferir: Apêndice C, item C6 (após toda a limpeza).
PY-14 N3 - Intermediário	Agregação nomeada: por marca, devolva n_pecas (contagem), impr_total (soma) e impr_media (média) em um único agg, com nomes de coluna limpos.
PY-15 N4 - Avançado	KPIs semanais: impressões por semana, variação percentual semana a semana (pct_change) e acumulado (cumsum). Use <code>pd.Grouper(freq='W')</code> ou <code>dt.to_period('W')</code>. Dica: Semanas sem dados aparecem? Precisam aparecer?
PY-16 N4 - Avançado	pivot_table com marcas nas linhas, tipos de mídia nas colunas, soma de impressões, fill_value=0 e totais nas margens (margins=True). Como conferir: A soma da coluna 'All' bate com o total geral de impressões.
PY-17 N4 - Avançado	CTR por marca considerando só DIGITAL e SOCIAL. Compare o resultado com o que você obterá incluindo TV e explique a diferença.

Bloco D - Organização, testes e exportação

PY-18 N4 - Avançado	Transforme a limpeza em funções puras: <code>normalizar_texto(serie)</code>, <code>parse_data(serie)</code>, <code>parse_inteiro(serie)</code>, <code>limpar_midia(df) -> DataFrame</code>. Garanta que a função não altera o DataFrame de entrada (use <code>copy()</code>). Dica: Teste: chame duas vezes com a mesma entrada. O resultado é igual? A entrada continua suja?
PY-19 N4 - Avançado	Organize em módulos: <code>etl/extract.py</code>, <code>etl/clean.py</code>, <code>etl/kpis.py</code> e <code>main.py</code>. Nenhum código deve rodar apenas ao ser importado (<code>if __name__ == '__main__':</code>). Como conferir: <code>python -m main</code> gera o resultado; <code>import etl.clean</code> não imprime nada.

PY-20 N4 - Avançado	Exporte para Excel com 2 abas (KPIs por marca e KPIs semanais) e para CSV com <code>encoding='utf-8-sig'</code> . Abra o CSV no Excel e confira os acentos. Rode também sem <code>utf-8-sig</code> e veja o que acontece.
PY-21 N4 - Avançado	Escreva ao menos 5 testes com <code>pytest</code> (normalização de texto, parse de data, parse de número, remoção de duplicatas, cálculo de CTR). Use <code>pytest.mark.parametrize</code> em pelo menos um. Dica: Teste com 3 a 5 linhas fabricadas à mão, nunca com o arquivo inteiro. Como conferir: <code>pytest -q</code> passa. Quebre uma função de propósito e veja o teste falhar.

Bloco E - Desafios integradores

PY-22 N5 - Desafio	Escreva <code>validar(df) -> DataFrame</code> que devolva uma tabela de problemas com as colunas <code>linha</code>, <code>coluna</code>, <code>severidade</code>, <code>mensagem</code>, no estilo da tela de revisão do portal. Regras: tipo de mídia no domínio; impressões e cliques ≥ 0; cliques $\leq$ impressões; data dentro da janela 01/01/2026 a 28/02/2026; marca existente no cadastro. A função não deve parar no primeiro erro. Dica: Você pode comparar com Pandera depois (<code>lazy=True</code>). Como conferir: Apêndice C, itens C4 (contagens esperadas por regra).
PY-23 N5 - Desafio	Mini ETL completo: Excel -> DataFrame limpo -> tabela <code>staging.media_item</code> no seu Postgres (via SQLAlchemy ou psycopg). Rode duas vezes: a segunda não pode duplicar linhas (idempotência). Registre quantas linhas entraram e quantas foram descartadas, e por quê. Dica: Combina com SQL-15 (ON CONFLICT). Como conferir: <code>SELECT count(*)</code> igual nas duas execuções (Apêndice C, item C5 traz o total esperado).
PY-24 N5 - Desafio	Desempenho: gere 500 mil linhas fictícias e some impressões por marca de três formas: laço com <code>iterrows</code>, <code>apply</code> e <code>groupby</code>. Meça com <code>time.perf_counter</code> e escreva a conclusão.

Parte 3 - PostgreSQL na prática

Por que importa: É onde mora a segurança que a RBS pediu como requisito extremo: schemas, permissões e, principalmente, Row-Level Security. O teste 'marca A tenta ler marca B' sai daqui.

Conceitos antes de começar

Schema é um namespace dentro do banco (`core.brand`, `dw.fact_media`). Organiza e permite dar permissões por área. O `search_path` define em qual schema o PostgreSQL procura nomes sem prefixo.

Role é usuário ou grupo (`LOGIN` ou `NOLOGIN`). Permissões são em camadas: para ler `core.brand` é preciso `USAGE` no schema e `SELECT` na tabela. O **dono** da tabela tem todos os direitos nela. `ALTER DEFAULT PRIVILEGES` cobre tabelas criadas no futuro.

View é uma consulta salva, calculada a cada uso. **View materializada** guarda o resultado e só muda com `REFRESH MATERIALIZED VIEW` (rápida para ler, pode estar desatualizada). Para o `REFRESH ... CONCURRENTLY` é necessário um índice único.

JSONB guarda documentos JSON em formato binário, consultáveis (`->`, `->>`, `@>`) e indexáveis (GIN). Bom para campos pontuais e flexíveis; ruim quando vira desculpa para fugir de modelar colunas.

Row-Level Security (RLS). Políticas que o banco aplica a cada linha. `ENABLE ROW LEVEL SECURITY` sem nenhuma policy significa **negar tudo**. `USING` decide quais linhas você enxerga (e quais pode alterar); `WITH CHECK` decide quais linhas novas ou alteradas são aceitas. O dono da tabela e superusers ignoram RLS, a não ser que se use `FORCE ROW LEVEL SECURITY` (e superusers e roles com `BYPASSRLS` sempre ignoram). O contexto do usuário chega por variáveis de sessão, ex.: `set_config('app.brand_ids', ..., true)` dentro da transação.

Leituras: postgresql.org/docs/16: 'Schemas', 'Privileges', 'Row Security Policies', 'Materialized Views', 'JSON Types'.

Bloco A - Schemas e roles

PG-01 N1 - Aquecimento	Crie os schemas <code>ref</code> , <code>core</code> , <code>staging</code> , <code>dw</code> e <code>audit</code> . Liste-os com o comando de barra do <code>psql</code> que mostra donos e permissões. Como conferir: Cinco schemas, com você como dono.
PG-02 N1 - Aquecimento	Crie uma tabela <code>teste</code> sem prefixo de schema. Em qual schema ela caiu? Descubra com <code>SHOW search_path</code> e com o catálogo. Ajuste o <code>search_path</code> da sua sessão e depois apague a tabela.
PG-03 N2 - Básico	Crie a role <code>app_readonly</code> com <code>LOGIN</code> . Conecte como ela e tente <code>SELECT</code> em <code>core.brand</code> : leia a mensagem. Corrija com <code>GRANT USAGE + GRANT SELECT</code> . Depois faça <code>REVOKE</code> e confirme que volta a falhar. Dica: Se der 'permission denied for schema', o problema é a camada do schema; se for 'for table', é a da tabela. Como conferir: Você deve conseguir provocar e corrigir os dois tipos de erro.
PG-04 N2 - Básico	Role de grupo: crie <code>analistas</code> (<code>NOLOGIN</code>), conceda <code>SELECT</code> a ela e adicione duas roles de login como membros. Teste herança. Revogue a participação de uma e veja o efeito.
PG-05 N3 - Intermediário	Monte, por conta própria, as roles do guia: <code>app_java</code> , <code>app_data</code> , <code>app_readonly</code> e <code>auditor</code> , aplicando mínimo privilégio (ver seção 3.8 do guia para o objetivo de cada uma, sem copiar o script). Preencha uma matriz role x schema x (SELECT, INSERT, UPDATE, DELETE) com o que deveria acontecer e depois teste cada célula. Registre as divergências. Como conferir: Nenhuma divergência entre a matriz esperada e a obtida no final.
PG-06 N3 - Intermediário	Crie uma tabela nova depois de dar <code>GRANT SELECT ON ALL TABLES</code> a <code>app_readonly</code> . Ela consegue ler a nova? Resolva com <code>ALTER DEFAULT PRIVILEGES</code> e explique por que o <code>ALL TABLES</code> não cobre o futuro.

Bloco B - Views, views materializadas e JSONB

PG-07 N2 - Básico	Crie <code>dw.v_impressoes_por_marca</code> (view) sobre as tabelas do Dataset A (ou do seu <code>core</code>). Insira uma veiculação nova e consulte a view de novo. Como conferir: A view reflete o novo dado imediatamente.
PG-08 N3 - Intermediário	Crie a versão materializada da mesma consulta. Insira dados novos e consulte: mudou? Rode <code>REFRESH</code> e consulte de novo. Com 1 milhão de linhas (SQL-32), compare o tempo de ler a view e a materializada. Como conferir: Antes do refresh a materializada continua com o valor antigo.
PG-09 N4 - Avançado	Rode <code>REFRESH MATERIALIZED VIEW CONCURRENTLY</code> . Se falhar, leia o erro, descubra o que falta e corrija. Explique qual vantagem o <code>CONCURRENTLY</code> traz para quem está lendo durante o refresh.
PG-10 N2 - Básico	Crie <code>core.report</code> com coluna <code>params jsonb</code> . Insira 3 relatórios com parâmetros diferentes (período, formato, seções). Consulte com <code>-></code> , <code>->></code> e o operador de contenção <code>@></code> .
PG-11 N3 - Intermediário	Crie um índice GIN em <code>params</code> , popule 200 mil linhas e use <code>EXPLAIN</code> para ver se uma busca com <code>@></code> o utiliza. Escreva 3 linhas sobre quando JSONB é má ideia no portal.
PG-12 N3 - Intermediário	Dado um array JSON de opções de pergunta de pesquisa (<code>options jsonb</code>), use <code>jsonb_array_elements</code> para transformá-lo em linhas, uma por opção.

Bloco C - Row-Level Security

PG-13 N2 - Básico	Crie <code>media_item</code> com <code>brand_id</code> , com dados de duas marcas. Ligue <code>ENABLE ROW LEVEL SECURITY</code> e consulte como uma role que não é dona da tabela. O que acontece? Depois consulte como dono. Explique a diferença. Como conferir: Role comum: 0 linhas e nenhum erro. Dono: todas as linhas.
PG-14 N3 - Intermediário	Crie as roles <code>marca_a</code> e <code>marca_b</code> e uma policy simples e fixa (cada role vê só as linhas da sua marca). Teste <code>SELECT</code> com cada uma. Como conferir: Cada role vê só as suas linhas.
PG-15 N3 - Intermediário	Contexto de sessão: troque a policy para usar <code>current_setting('app.brand_ids', true)</code> . Faça <code>SET</code> e <code>SET LOCAL</code> dentro de uma transação e explique a diferença ao fazer <code>COMMIT</code> . Por que <code>SET LOCAL</code> (ou <code>set_config(..., true)</code>) é o correto com pool de conexões? Dica: Em pool, a mesma conexão atende usuários diferentes em momentos diferentes.
PG-16 N4 - Avançado	Escreva, por conta própria, a policy completa: tipo <code>ADMIN_RBS</code> vê tudo; <code>ADMIN_PATROCINADOR</code> vê só as marcas listadas em <code>app.brand_ids</code> (lista separada por vírgula, convertida para <code>uuid[]</code>). Teste 4 cenários: RBS, patrocinador com 1 marca, patrocinador com 2 marcas e sem contexto . Dica: O guia (seção 3.3.9) mostra o formato geral; tente escrever primeiro sem olhar. Como conferir: Sem contexto: 0 linhas, sem erro. É o comportamento seguro.
PG-17 N4 - Avançado	<code>WITH CHECK</code> : como <code>marca_a</code> , tente inserir uma linha com <code>brand_id</code> da marca B. Teste com policy só com <code>USING</code> e com <code>WITH CHECK</code> explícito. Anote o que muda em <code>FOR ALL</code> e em <code>FOR INSERT</code> .
PG-18 N4 - Avançado	A armadilha do dono: conecte como dono da tabela e veja que a RLS não se aplica. Resolva com <code>FORCE ROW LEVEL SECURITY</code> . Depois teste com um superuser e com uma role <code>BYPASSRLS</code> . Conclua: que regras a role da aplicação deve cumprir? Como conferir: Regra: não ser dona das tabelas, não ser superuser, não ter <code>BYPASSRLS</code> .
PG-19 N4 - Avançado	Tente <code>UPDATE</code> e <code>DELETE</code> em linhas de outra marca, como <code>marca_a</code> . Por que não dá erro e sim '0 rows'? O que isso diz sobre o que o usuário consegue descobrir sobre dados que não são dele?
PG-20 N5 - Desafio	Teste de isolamento automatizado: um script (Python ou SQL) que, para cada marca, abre uma transação com o contexto dela, confere que o <code>count</code> bate com o esperado e que o <code>count</code> de qualquer outra marca é 0. Rode em todas as tabelas com <code>brand_id</code> . Quebre a policy de propósito e veja o teste falhar. Dica: Esse é o precursor do teste que a banca mais vai valorizar. Como conferir: Teste passa com a policy correta e falha com a policy quebrada.

PG-21

N5 - Desafio

Desempenho com RLS: compare **EXPLAIN ANALYZE** de uma consulta com e sem a policy, com e sem índice em **brand_id**. Onde a condição da policy aparece no plano? O índice continua sendo usado?

PG-22

N5 - Desafio

Migrações versionadas: organize tudo que você fez nesta parte em **V001__...sql**, **V002__...sql**, etc., e crie um script **reset** que recria o banco do zero aplicando-os em ordem, incluindo roles, RLS e dados de exemplo. Rode em outra máquina ou pasta.

Dica: É o hábito que o Flyway/Alembic automatiza.

Como conferir: Banco recriado do zero e teste PG-20 passando, com um único comando.

Parte 4 - APIs REST, FastAPI, Pydantic e OpenAPI

Por que importa: É o contrato entre a sua trilha e a trilha Java. O OpenAPI v0.1 precisa estar assinado pelas duas equipes ainda na Sprint 1, então ler e escrever esse contrato é prioridade.

Conceitos antes de começar

REST. Recursos são substantivos na URL (`/v1/brands/{id}/kpis`); o verbo HTTP diz a ação (GET lê, POST cria, PUT/PATCH altera, DELETE remove). A API é **stateless**: cada requisição carrega tudo de que precisa. Códigos de status comunicam o resultado: 200 OK, 201 criado, 202 aceito (processando), 400/422 entrada inválida, 401 não autenticado, 403 autenticado sem permissão, 404 não encontrado.

Pydantic. Modelos que declaram o formato esperado dos dados e validam/convertem automaticamente. Se a entrada não bate, a validação falha com uma lista de erros em vez de deixar o dado ruim entrar.

FastAPI. Usa os **type hints** e os modelos Pydantic para validar requisições, serializar respostas e gerar a documentação **OpenAPI** sozinho (`/docs` e `/openapi.json`). O servidor que a executa é o **uvicorn**.

OpenAPI. Um arquivo (YAML/JSON) que descreve rotas, parâmetros, corpos e respostas. Em **contract-first** você escreve o contrato antes do código e as duas equipes trabalham em paralelo contra mocks. Reaproveitamento por `$ref` para `components/schemas`.

Injeção de dependências (`Depends`): funções reutilizáveis que o FastAPI executa antes do endpoint (conexão com banco, usuário atual). É por onde passam autenticação e contexto de marca.

Tarefas longas não bloqueiam a requisição: **POST** devolve **202** com um `job_id`, e o cliente consulta **GET** `/jobs/{id}` até terminar.

Leituras: fastapi.tiangolo.com/tutorial (siga a ordem até 'Dependencies' e 'Handling Errors'); spec.openapis.org/oas/v3.1.0; docs.pydantic.dev. Seção 3.7.3 do guia técnico mostra o formato do contrato do projeto.

Bloco A - HTTP e primeiros endpoints

API-01 N1 - Aquecimento	HTTP na mão: use <code>curl -v</code> (ou Bruno/Postman) em uma API pública que devolva JSON. Identifique o método, a URL, o status, 3 cabeçalhos de resposta e o corpo. Repita com um recurso inexistente e compare o status.
API-02 N1 - Aquecimento	Crie um app FastAPI com <code>GET /health</code> devolvendo <code>{"status": "ok"}</code> . Rode com <code>uvicorn --reload</code> , abra <code>/docs</code> e <code>/openapi.json</code> . Como conferir: Swagger UI abre e lista a rota.
API-03 N2 - Básico	Crie <code>GET /v1/brands/{brand_id}/kpis</code> devolvendo KPIs fictícios (um dicionário em memória). Tipe <code>brand_id</code> como <code>UUID</code> . Chame com um UUID inválido: qual status vem e por quê? Como conferir: 422 com a descrição do campo inválido.
API-04 N2 - Básico	Parâmetro de consulta <code>period</code> com <code>Enum</code> (<code>parcial</code> ou <code>final</code>) e valor padrão. Teste um valor fora do enum e veja a mensagem.
API-05 N2 - Básico	Defina modelos Pydantic <code>Kpi</code> (nome, valor, unidade, variação opcional) e <code>KpiSet</code> . Use <code>response_model</code> . Retorne propositalmente um dicionário com um campo a mais e veja que ele não vaza na resposta. Dica: Esse comportamento é importante para não expor dados sensíveis por acidente.
API-06 N2 - Básico	Marca desconhecida: devolva 404 com <code>HTTPException</code> e uma mensagem útil. Qual a diferença entre 404, 403 e 401 neste contexto? Por que, em segurança, às vezes se devolve 404 em vez de 403? Dica: Pense em IDOR: confirmar que o recurso existe já é um vazamento.

Bloco B - Validação e estrutura

API-07 N3 - Intermediário	<code>POST /v1/media-items</code> com corpo validado: <code>media_type</code> do enum, <code>aired_date</code> data, <code>impressions</code> e <code>clicks</code> com <code>Field(ge=0)</code> . Responda 201 . Teste 3 corpos inválidos diferentes e leia as mensagens de erro de cada um.
-------------------------------------	--

API-08 N3 - Intermediário	Validadores customizados: (a) <code>cnpj</code> com verificação do dígito verificador (implemente o algoritmo você mesmo); (b) <code>clicks <= impressions</code> em um validador de modelo (<code>model_validator</code>). Dica: Pesquise o algoritmo do dígito verificador do CNPJ e teste com um CNPJ válido e um inválido.
API-09 N3 - Intermediário	Separe o app em <code>routers/kpis.py</code> , <code>schemas.py</code> e <code>services.py</code> e monte com <code>include_router</code> e o prefixo <code>/v1</code> . A lógica de cálculo não deve ficar dentro da função da rota. Como conferir: <code>/docs</code> continua igual; o arquivo <code>main.py</code> fica pequeno.
API-10 N3 - Intermediário	Dependência <code>current_user</code> : leia o cabeçalho <code>Authorization: Bearer ...</code> (por enquanto sem validar assinatura, só presença e formato). Sem o cabeçalho, devolva 401. Use essa dependência nas rotas de KPIs. Dica: A validação real do JWT é da Prioridade 3; aqui o objetivo é entender <code>Depends</code> .
API-11 N3 - Intermediário	Erros padronizados: registre um <code>*exception handler*</code> que devolva <code>application/problem+json</code> (RFC 7807) com <code>type</code> , <code>title</code> , <code>status</code> e <code>detail</code> . Aplique também aos erros de validação.
API-12 N3 - Intermediário	Leia o <code>openapi.json</code> que o FastAPI gerou e responda por escrito: onde ficam os schemas? Como o enum aparece? O que é <code>operationId</code> ? Como um <code>\$ref</code> é resolvido?

Bloco C - Contract-first, jobs e testes

API-13 N4 - Avançado	Escreva à mão um <code>openapi.yaml</code> (versão 3.1) com <code>GET /v1/brands/{brand_id}/kpis</code> e <code>POST /v1/reports</code> (resposta 202 com o schema <code>Job</code>). Valide no editor.swagger.io ou com <code>openapi-spec-validator</code> . Dica: Use a seção 3.7.3 do guia só para conferir a estrutura depois de tentar. Como conferir: O validador não aponta erros.
API-14 N4 - Avançado	Suba um servidor <code>*mock*</code> do seu contrato com Prism (<code>stoplight/prism</code>) e consuma com <code>curl</code> . Simule o trabalho do time Java, que desenvolve contra o mock enquanto você ainda não implementou nada.
API-15 N4 - Avançado	Implemente o contrato em FastAPI. Compare o <code>openapi.json</code> gerado com o seu YAML e liste as divergências (nomes, tipos, códigos de status). Corrija o código ou o contrato, e justifique qual dos dois mudou. Como conferir: Nenhuma divergência relevante entre YAML e JSON gerado.
API-16 N4 - Avançado	Jobs assíncronos: <code>POST /v1/reports</code> devolve 202 com <code>job_id</code> ; <code>GET /v1/jobs/{id}</code> passa por <code>queued</code> , <code>running</code> e <code>done</code> (use <code>BackgroundTasks</code> , um dicionário em memória e <code>time.sleep</code> simulando o trabalho). Faça polling com <code>curl</code> . O que se perde se o servidor reiniciar? Como uma tabela <code>job</code> resolveria?
API-17 N4 - Avançado	Testes com <code>TestClient/httpx</code> e <code>pytest</code> : para cada endpoint cubra sucesso, 404, 422 e 401. Use fixtures. Como conferir: Todos passam; quebre uma rota de propósito e veja o teste falhar.

Bloco D - Integração com banco e contexto de marca

API-18 N5 - Desafio	Ligue o endpoint de KPIs ao PostgreSQL: leia da view do PG-07 usando SQLAlchemy 2 ou psycopg. Abra e feche a conexão por requisição com uma dependência (<code>yield</code>). Como conferir: Os números da API batem com a consulta SQL feita direto no banco.
API-19 N5 - Desafio	Propague o contexto de segurança: na dependência, abra uma transação e execute <code>set_config('app.user_type', ...)</code> e <code>set_config('app.brand_ids', ...)</code> com valores vindos de um token fictício. Prove, com 2 chamadas, que o usuário da marca A não vê dados da B pela API . Dica: Reaproveita as policies do PG-16 e o teste do PG-20. Como conferir: Chamada com a marca A devolve só dados de A; sem contexto devolve vazio.

API-20

N5 - Desafio

Rode o Schemathesis contra sua API (`schemathesis run` apontando para o `openapi.json`). Corrija tudo que ele encontrar. Anote os 3 tipos de falha mais interessantes no diário.

Como conferir: A execução termina sem falhas.

API-21

N5 - Desafio

Versionamento: introduza `/v2` com uma mudança incompatível (renomear um campo) mantendo `/v1` funcionando. Escreva, em 5 linhas, a regra que diferencia uma mudança compatível de uma incompatível.

Parte 5 - Git e Docker Compose

Por que importa: A integração com a trilha Java só funciona se todo mundo versionar do mesmo jeito e subir o mesmo ambiente com **docker compose up**. O guia lista isso como pré-requisito da Sprint 1.

Conceitos antes de começar

Git guarda um grafo de **commits** (fotografias do projeto). Seu trabalho passa por três áreas: diretório de trabalho, **staging** (**git add**) e repositório (**git commit**). Um **branch** é só um ponteiro para um commit. **merge** junta históricos preservando-os; **rebase** reaplica seus commits sobre outra base, deixando o histórico linear.

Desfazer depende de onde está o erro: **restore** (diretório/staging), **reset** (move o branch, reescreve histórico local), **revert** (cria um commit novo que desfaz outro, seguro para histórico já publicado). Segredos commitados continuam no histórico mesmo depois de apagados do arquivo.

Docker. **Imagem** é o molde (somente leitura); **container** é a instância em execução. Dados dentro do container somem com ele, a menos que estejam em um **volume**. Cada instrução do **Dockerfile** cria uma **camada** em cache: a ordem das instruções afeta a velocidade do build.

Compose descreve vários serviços (banco, API, storage) em um YAML, com rede própria: um serviço enxerga o outro pelo **nome do serviço** (**db**), não por **localhost**. **healthcheck + depends_on: condition: service_healthy** garantem a ordem de inicialização. Senhas ficam em **.env**, fora do Git.

Leituras: git-scm.com/book/pt-br/v2 (capítulos 1 a 3); docs.docker.com/get-started e docs.docker.com/compose; conventionalcommits.org.

Bloco A - Git

GD-01 N1 - Aquecimento	Crie um repositório do zero, configure nome e e-mail e faça o primeiro commit com um README.md . A cada passo rode git status , git diff e git log --oneline e descreva o que mudou.
GD-02 N1 - Aquecimento	Crie um .gitignore para Python (ambiente virtual, __pycache__ , .env , arquivos de dados reais). Crie um .env e prove que o Git não o rastreia (git status , git check-ignore -v).
GD-03 N2 - Básico	Branches: crie feat/ia-seed , faça 3 commits e junte na main (primeiro com <i>*fast-forward*</i> ; depois repita com a main tendo recebido um commit diferente para forçar um <i>*merge commit*</i>). Visualize com git log --graph --oneline --all . Como conferir: O grafo mostra as duas formas de merge.
GD-04 N2 - Básico	Conventional Commits: reescreva 5 mensagens ruins (como 'arrumei', 'mudanças', 'wip') no padrão feat: , fix: , docs: , chore: , test: , cada uma com escopo e descrição curta.
GD-05 N3 - Intermediário	Provoque um conflito de merge de propósito (duas branches alterando a mesma linha). Resolva manualmente, entenda os marcadores e conclua o merge.
GD-06 N3 - Intermediário	Desfazendo: monte cenários para git restore , git stash , git reset --soft/--mixed e git revert . Escreva uma tabelinha: quando usar cada um e qual é perigoso em histórico já enviado ao remoto.
GD-07 N3 - Intermediário	Remoto: publique no GitHub, clone em outra pasta (simulando um colega), abra um Pull Request e faça uma revisão cruzada com outra pessoa (ou uma segunda conta). Pratique comentar, pedir mudança e aprovar.
GD-08 N4 - Avançado	Rebase interativo: crie 4 commits 'wip' seguidos e junte (<i>*squash*</i>) em um com mensagem boa. Compare o histórico com o de um merge normal. Dica: Só reescreva histórico que ainda não foi compartilhado.

GD-09 N4 - Avançado	Crie um workflow do GitHub Actions que rode ruff e pytest a cada push e Pull Request. Faça um PR com teste quebrado e veja o CI bloquear. Configure a main como protegida. Como conferir: O PR com teste falhando não pode ser mesclado.
GD-10 N4 - Avançado	Segredo vazado (em repositório de teste!): commite uma senha falsa, apague no commit seguinte e mostre que continua no histórico. Remova do histórico (git filter-repo ou recriação do repositório) e explique por que, na vida real, a senha precisa ser trocada . Configure pre-commit com detect-secrets para impedir o commit.

Bloco B - Docker e Docker Compose

GD-11 N1 - Aquecimento	Instale o Docker, rode hello-world , depois docker ps -a , docker images , docker rm , docker rmi . Explique em uma frase a diferença entre imagem e container.
GD-12 N1 - Aquecimento	Rode o PostgreSQL 16 com volume nomeado , crie uma tabela, apague o container e suba outro com o mesmo volume. Os dados persistem? Repita sem volume. Como conferir: Com volume: dados persistem. Sem: somem.
GD-13 N2 - Básico	Escreva um Dockerfile para o seu app FastAPI (base python:3.12-slim , instalar dependências, rodar uvicorn). Construa, rode com mapeamento de porta e acesse /docs . Explique EXPOSE x -p .
GD-14 N2 - Básico	Cache de camadas: meça o tempo de rebuild após mudar um arquivo .py em duas versões do Dockerfile: (a) COPY . . antes de instalar as dependências; (b) copiando requirements.txt e instalando antes de copiar o código. Explique a diferença. Como conferir: A versão (b) reconstrói bem mais rápido.
GD-15 N3 - Intermediário	Escreva um docker-compose.yml com dois serviços, db (Postgres) e api (seu FastAPI). Credenciais via arquivo .env . A API deve conectar usando o host db . Por que localhost não funciona dentro do container da API? Como conferir: docker compose up sobe os dois e a API consulta o banco.
GD-16 N3 - Intermediário	Adicione healthcheck ao banco (pg_isready) e depends_on com condition: service_healthy na API. Derrube e suba e acompanhe a ordem nos logs. O que acontece sem o healthcheck?
GD-17 N3 - Intermediário	Inicialização automática: monte uma pasta db/init com seus arquivos SQL (schemas, roles, seed) em /docker-entrypoint-initdb.d . Suba do zero com docker compose down -v && docker compose up . Quando esses scripts rodam e quando não rodam? Dica: Dica: só na primeira inicialização de um volume vazio. Como conferir: Banco criado com schemas e dados de exemplo sem nenhum passo manual.
GD-18 N4 - Avançado	Adicione o MinIO (storage compatível com S3) ao compose, crie um bucket e envie um arquivo pela CLI (mc) ou por boto3 . Garanta que os dados do MinIO também usem volume.
GD-19 N4 - Avançado	Redes: defina duas redes (backend e public). O banco não publica porta no host; só a API publica. Prove que o psql do seu computador não alcança o banco, mas a API sim. Relacione com o guia: 'sem acesso público à porta 5432'.
GD-20 N5 - Desafio	Meta de entrega: em uma pasta limpa (um git clone novo, sem nada instalado além do Docker), docker compose up sobe banco com seed, API e MinIO. Escreva um README com quick start em 3 comandos e peça a uma pessoa para seguir só ele, sem ajuda. Anote onde ela travou e corrija o README. Como conferir: Outra pessoa chega a /docs funcionando só com o README.

Parte 6 - Mini-projeto integrador (Semana 1)

O seu plano diz para fazer logo na Semana 1 um mini-projeto de ponta a ponta. Ele junta as partes anteriores e mostra onde estão suas lacunas. Cada etapa tem um critério de pronto; só avance quando ele for verdadeiro.

Etapa	O que fazer e critério de pronto
1. Dados de entrada	Parta do <code>midia_ficticia.xlsx</code> . Critério: o arquivo é lido sem erro e você tem a lista dos problemas encontrados (PY-03).
2. Limpeza testada	Módulo de limpeza com funções puras e 5 testes passando (PY-18 a PY-21). Critério: <code>pytest</code> verde e relatório de problemas (PY-22).
3. Banco	Schemas <code>ref</code> , <code>core</code> , <code>staging</code> e <code>dw</code> , tabelas e constraints versionadas em arquivos <code>V###__*.sql</code> (SQL-14, PG-22). Critério: banco recriado do zero com um comando.
4. Carga idempotente	Excel limpo vai para <code>staging.media_item</code> e depois para <code>core.media_item</code> com <code>ON CONFLICT</code> (PY-23, SQL-15). Critério: rodar duas vezes não duplica.
5. Isolamento	RLS ligada com policy por marca e o teste automatizado (PG-16, PG-20). Critério: teste verde; e vermelho se a policy for quebrada.
6. KPI	Uma view materializada com impressões por marca e semana (PG-08, SQL-28). Critério: números conferem com o pandas.
7. API	<code>GET /v1/brands/{id}/kpis</code> lendo dessa view, com contexto de marca (API-18, API-19). Critério: usuário A não vê dados da marca B.
8. Ambiente	Tudo sobe com <code>docker compose up</code> e há um README de 3 comandos (GD-20). Critério: outra pessoa consegue rodar.

Extensão (Prioridade 2): um gráfico com Plotly exportado em PNG e um PDF simples com capa, tabela e gráfico gerado com Jinja2 + WeasyPrint. Isso já é a Sprint 2, mas faz uma boa fatia vertical 'do dado à tela'.

Parte 7 - Autoavaliação conceitual

Tente responder cada pergunta em voz alta ou por escrito, em até 4 linhas, **sem consultar**. Se travar, volte ao bloco indicado. Várias delas são o tipo de pergunta que a banca pode fazer.

#	Pergunta	Se travar, revise
1	Qual a diferença entre <code>WHERE</code> e <code>HAVING</code> ?	SQL-23
2	Quando um <code>LEFT JOIN</code> é necessário em vez de <code>INNER JOIN</code> ? Dê um exemplo do portal.	SQL-21
3	Por que <code>COUNT(*)</code> e <code>COUNT(coluna)</code> podem diferir?	SQL-20
4	Explique a 3FN com um exemplo seu e o que ela evita.	SQL-05, SQL-06
5	Por que modelamos usuário x marca como tabela associativa?	SQL-04
6	O que uma carga idempotente garante e como o banco ajuda?	SQL-15, PY-23
7	Por que <code>NOT IN</code> pode dar resultado inesperado com <code>NULL</code> ?	SQL-22
8	O que é uma função pura e por que ela torna o ETL testável?	PY-18
9	Por que a ordem importa entre normalizar texto e remover duplicatas?	PY-11
10	Qual a diferença entre <code>int64</code> e <code>Int64</code> no pandas?	PY-09

#	Pergunta	Se travar, revise
11	Por que RLS no banco e não só <code>WHERE brand_id = ?</code> na aplicação?	PG-13 a PG-18
12	O que acontece quando uma tabela tem RLS ligada e nenhuma policy?	PG-13
13	Por que o dono da tabela e o superuser 'furam' a RLS? Como a aplicação evita isso?	PG-18
14	Qual a diferença entre <code>SET</code> e <code>SET LOCAL</code> , e por que isso importa com pool de conexões?	PG-15
15	Quando usar view e quando usar view materializada?	PG-07, PG-08
16	Qual a diferença entre 401, 403 e 404 e por que às vezes se escolhe 404 por segurança?	API-06
17	Por que escrever o contrato OpenAPI antes do código?	API-13, API-14
18	Por que o <code>POST /reports</code> devolve 202 em vez de 200?	API-16
19	Qual a diferença entre <code>merge</code> e <code>rebase</code> , e quando o rebase é perigoso?	GD-08
20	Por que apagar uma senha do arquivo não resolve se ela foi commitada?	GD-10
21	Por que dentro do Compose usamos <code>db</code> como host e não <code>localhost</code> ?	GD-15

Modelo de diário de erros

Uma linha por erro. Escreva à mão ou em uma planilha; o ponto é transformar o erro em aprendizado.

Data / exercício	O que eu esperava que acontecesse	O que realmente aconteceu (mensagem de erro, se houver)	Causa real	O que vou lembrar da próxima vez

Apêndice A - Dataset A (script de carga para os exercícios SQL)

Rode no **psql** ou no DBeaver em um banco de testes. Tudo vai para o schema **lab**, para não se misturar com o seu **core**. Os dados são fictícios e têm casos de propósito: marcas **sem mídia**, linhas **duplicadas**, **nulos** em cliques e impressões e uma marca que passa de 90% de uso de cortesias. Este script cria as tabelas para você; modelar as suas próprias é o que a Parte 1, bloco B pede.

```
CREATE SCHEMA IF NOT EXISTS lab;
SET search_path TO lab;

CREATE TABLE tier (id smallint PRIMARY KEY, code text UNIQUE, name text);
CREATE TABLE brand (id int PRIMARY KEY, name text, tier_id smallint REFERENCES tier(id), sector text);
CREATE TABLE media_item (
  id int PRIMARY KEY, brand_id int REFERENCES brand(id), media_type text,
  vehicle text, title text, aired_date date, impressions bigint, clicks bigint);
CREATE TABLE courtesy (id int PRIMARY KEY, brand_id int REFERENCES brand(id), area text,
  qty_contracted int, qty_used int);

INSERT INTO tier VALUES
(1, 'MASTER', 'Master'),
(2, 'SUBCOTA_G', 'Subcota G'),
(3, 'SUBCOTA', 'Subcota');

INSERT INTO brand VALUES
(1, 'Alfa Varejo', 1, 'Varejo'),
(2, 'Beta Bebidas', 1, 'Bebidas'),
(3, 'Gama Bank', 1, 'Financeiro'),
(4, 'Delta Telecom', 2, 'Telecom'),
(5, 'Epsilon Cosméticos', 2, 'Cosméticos'),
(6, 'Zeta Foods', 3, 'Alimentos'),
(7, 'Eta Seguros', 3, 'Seguros'),
(8, 'Theta Sports', 3, 'Esportes'),
(9, 'Iota Moda', 3, 'Moda'),
(10, 'Kappa Tech', 3, 'Tecnologia');

INSERT INTO media_item VALUES
(1, 1, 'TV', 'RBS TV', 'Spot Abertura', '2026-01-05', 1200000, NULL),
(2, 1, 'DIGITAL', 'Instagram', 'Carrossel Looks', '2026-01-06', 450000, 9800),
(3, 1, 'SOCIAL', 'TikTok', 'Desafio Verão', '2026-01-12', 800000, 21000),
(4, 1, 'RADIO', 'Atlântida FM', 'Jingle 30s', '2026-01-13', 300000, NULL),
(5, 1, 'DIGITAL', 'Site Portal', 'Banner Home', '2026-01-20', 600000, 7200),
(6, 2, 'TV', 'RBS TV', 'Spot Brinde', '2026-01-05', 2000000, NULL),
(7, 2, 'DIGITAL', 'YouTube', 'Pré-roll Festival', '2026-01-08', 900000, 15000),
(8, 2, 'SOCIAL', 'Instagram', 'Stories Backstage', '2026-01-15', 500000, 12000),
(9, 2, 'DIGITAL', 'Instagram', 'Reels Line-up', '2026-01-22', 700000, 18500),
(10, 3, 'RADIO', 'Atlântida FM', 'Spot Cashback', '2026-01-07', 250000, NULL),
(11, 3, 'DIGITAL', 'Site Portal', 'Banner Cartão', '2026-01-14', 320000, 4100),
(12, 3, 'IMPRESSO', 'Jornal ZH', 'Anúncio Página', '2026-01-19', 150000, NULL),
(13, 4, 'DIGITAL', 'Instagram', 'Post 5G', '2026-01-09', 280000, 5200),
(14, 4, 'SOCIAL', 'TikTok', 'Trend Sinal', '2026-01-16', 410000, 9900),
(15, 4, 'TV', 'RBS TV', 'Selo Patrocínio', '2026-01-21', 950000, NULL),
(16, 5, 'DIGITAL', 'Instagram', 'Tutorial Make', '2026-01-10', 220000, 6100),
(17, 5, 'SOCIAL', 'TikTok', 'Get Ready', '2026-01-18', 360000, 8800),
(18, 6, 'DIGITAL', 'Site Portal', 'Banner Receita', '2026-01-11', 90000, 1300),
(19, 6, 'RADIO', 'Atlântida FM', 'Menção Locutor', '2026-01-17', 120000, NULL),
(20, 7, 'DIGITAL', 'Instagram', 'Post Seguro Viagem', '2026-01-12', 75000, 900),
(21, 7, 'IMPRESSO', 'Jornal ZH', 'Anúncio Rodapé', '2026-01-23', NULL, NULL),
(22, 8, 'SOCIAL', 'Instagram', 'Collab Esportes', '2026-01-14', 130000, 3300),
(23, 8, 'DIGITAL', 'YouTube', 'Vlog Arena', '2026-01-24', 210000, 4000),
(24, 1, 'DIGITAL', 'Site Portal', 'Banner Home', '2026-01-20', 600000, 7200),
(25, 2, 'OOH', 'Outdoor Orla', 'Painel Orla', '2026-01-10', 500000, NULL),
(26, 4, 'DIGITAL', 'Instagram', 'Post 5G', '2026-01-09', 280000, 5200);

INSERT INTO courtesy VALUES
(1, 1, 'Arena', 100, 95),
(2, 1, 'Camarote', 40, 40),
(3, 2, 'Arena', 100, 60),
(4, 2, 'Premium', 20, 18),
(5, 3, 'Arena', 80, 72),
(6, 4, 'Arena', 50, 20),
(7, 4, 'Lounge', 10, 10),
(8, 5, 'Arena', 40, 10),
(9, 6, 'Arena', 20, 19),
(10, 7, 'Arena', 20, 5),
```

```
(11, 8, 'Arena', 15, 15);
```

Apêndice B - Resultados esperados (SQL)

Só consulte depois de tentar. Aqui estão os resultados, não as consultas. Se o seu número diferir, investigue o porquê (join errado, NULL, duplicata?) em vez de ajustar até bater.

B1 (SQL-17) - ids das linhas de `media_item`.

Filtro	ids
(a) tipo DIGITAL	2, 5, 7, 9, 11, 13, 16, 18, 20, 23, 24, 26
(b) entre 10 e 17/01/2026	3, 4, 8, 11, 14, 16, 18, 19, 20, 22, 25
(c) clicks nulos	9 linhas: 1, 4, 6, 10, 12, 15, 19, 21, 25
(d) clicks > 5000	2, 3, 5, 7, 8, 9, 13, 14, 16, 17, 24, 26

B2 (SQL-19) - marcas por cota e impressões por tipo (com as duplicatas, como a tabela está).

Cota	Marcas
Master	3
Subcota G	2
Subcota	5

Tipo	Impressões
DIGITAL	4.725.000
TV	4.150.000
SOCIAL	2.200.000
RADIO	670.000
OOH	500.000
IMPRESSO	150.000

B3 (SQL-20) - `COUNT(*)` = 26; `COUNT(clicks)` = 17; `COUNT(impressions)` = 25; `COUNT(DISTINCT brand_id)` = 8.

B4 (SQL-21 e SQL-22) - marcas sem nenhuma veiculação: Iota Moda, Kappa Tech. O `LEFT JOIN` completo devolve 10 linhas.

B5 (SQL-24) - CTR em % só de DIGITAL e SOCIAL (soma de cliques / soma de impressões, com as duplicatas):

Marca	CTR (%)
Alfa Varejo	1,84
Beta Bebidas	2,17
Gama Bank	1,28
Delta Telecom	2,09
Epsilon Cosméticos	2,57
Zeta Foods	1,44
Eta Seguros	1,2
Theta Sports	2,15

B6 (SQL-25) - % de cortesias usadas por marca e combinações com 90% ou mais:

Marca	% usado
Alfa Varejo	96,4
Beta Bebidas	65,0
Gama Bank	90,0

Delta Telecom	50,0
Epsilon Cosméticos	25,0
Zeta Foods	95,0
Eta Seguros	25,0
Theta Sports	100,0

Marca	Área	% usado
Alfa Varejo	Arena	95,0
Alfa Varejo	Camarote	100,0
Beta Bebidas	Premium	90,0
Gama Bank	Arena	90,0
Delta Telecom	Lounge	100,0
Zeta Foods	Arena	95,0
Theta Sports	Arena	100,0

B7 (SQL-26) - média dos totais por marca (8 marcas com média): 1.549.375. Acima da média: Beta Bebidas (4.600.000); Alfa Varejo (3.950.000); Delta Telecom (1.920.000).

B8 (SQL-27) - total de impressões por marca, na ordem cota, posição:

Cota	Posição	Marca	Impressões
Master	1	Beta Bebidas	4.600.000
Master	2	Alfa Varejo	3.950.000
Master	3	Gama Bank	720.000
Subcota G	1	Delta Telecom	1.920.000
Subcota G	2	Epsilon Cosméticos	580.000
Subcota	1	Theta Sports	340.000
Subcota	2	Zeta Foods	210.000
Subcota	3	Eta Seguros	75.000

B9 (SQL-28) - impressões por semana (semana começando na segunda-feira) e acumulado:

Semana	Impressões	Acumulado
2026-01-05	6.170.000	6.170.000
2026-01-12	3.015.000	9.185.000
2026-01-19	3.210.000	12.395.000

B10 (SQL-29) - grupos duplicados: 2 (marcas 1, 4). Linhas após a remoção: 24. Top 3 de impressões **sem** duplicatas: Beta Bebidas (4.600.000); Alfa Varejo (3.350.000); Delta Telecom (1.640.000). Com duplicatas: Beta Bebidas (4.600.000); Alfa Varejo (3.950.000); Delta Telecom (1.920.000).

B11 (SQL-31) - marca 2 (Beta Bebidas), impressões por tipo: DIGITAL 1.600.000; OOH 500.000; SOCIAL 500.000; TV 2.000.000. Tipos sem veiculação aparecem como 0 ou NULL, conforme sua escolha.

Apêndice C - Resultados esperados (pandas, midia_ficticia.xlsx)

Só consulte depois de tentar. Ordem de limpeza usada para chegar nestes números: (1) normalizar texto; (2) converter datas e números; (3) transformar impressões negativas em nulo; (4) descartar linhas sem data válida; (5) remover duplicatas pela chave (**marca, data, veículo, título**), mantendo a primeira ocorrência.

C1 (PY-02) - com o cabeçalho na linha certa, a aba Midia tem **59 linhas** e 8 colunas.

C2 (PY-03) - problemas que existem de propósito no arquivo:

- Título do relatório e linha em branco acima do cabeçalho (o cabeçalho real está na 3ª linha).
- Marca com caixa diferente e espaços sobrando (28 grafias para 8 marcas).
- Tipo de mídia com 13 variações (caixa, espaços, acento em Rádio) para 6 tipos reais.
- Data de veiculação em três formatos (objeto de data, dd/mm/aaaa, aaaa-mm-dd).
- Datas inválidas (31/02/2026 e sem data) e datas fora da janela da edição.
- Impressões como texto com ponto de milhar em 10 linhas; 1 nula; 1 negativa.
- Uma linha com cliques maiores do que impressões e uma com alcance nulo.
- Duplicatas: 2 idênticas e 2 que só aparecem depois de normalizar a marca.
- Cliques vazios em TV, rádio, impresso e OOH. **Isto não é erro:** são meios sem clique.

C3 (PY-06 e PY-07) - tipos de mídia: 13 valores brutos viram 6 (DIGITAL, IMPRESSO, OOH, RADIO, SOCIAL, TV). Marcas: 28 grafias viram 8.

C4 (PY-08, PY-10, PY-22) - contagens por regra:

Regra	Linhas
Datas inválidas (viram NaT)	2
Impressões como texto	10
Impressões nulas originais	1
Impressões negativas	1
Cliques maiores que impressões (após negativo virar nulo)	1
Data fora da janela 01/01 a 28/02/2026	2

C5 (PY-11 e PY-23) - após descartar datas inválidas: **57 linhas**. Duplicatas removidas: **4**. Linhas finais: **53**. Esse é o total que deve estar em **staging.media_item**, e não deve mudar na segunda execução.

C6 (PY-13) - total de impressões final: **47.474.000**.

Tipo	Impressões
DIGITAL	14.913.000
IMPRESSO	10.411.000
TV	9.825.000
RADIO	5.541.000
OOH	5.397.000
SOCIAL	1.387.000
Top 3 marcas	Impressões
Beta Bebidas	9.212.000
Eta Seguros	7.529.000
Theta Sports	7.369.000

Marca	Linhas finais
Beta Bebidas	8
Delta Telecom	8
Theta Sports	8
Epsilon Cosméticos	7
Gama Bank	6
Eta Seguros	6
Alfa Varejo	5
Zeta Foods	5

Se alguma contagem sua divergir, a causa mais comum é a **ordem** das etapas (por exemplo, remover duplicatas antes de normalizar a marca). Anote no diário de erros qual foi.